

Criterion C: Development

Techniques

- Class structure

Since my database revolves around pastries, I made a class and a format that each pastry follows. Whenever a pastry is called, it is given objects automatically and the objects are called on by the program. This reduces abstraction by having the program look for the same things every time a pastry is called.

```
package com.codebind;

import java.io.File;
import java.io.FileReader;
import java.util.Scanner;

13 usages
public class pastry {
    // initializes object
    6 usages
    String Pname;
    3 usages
    String information;
    4 usages
    String ingredients;
    5 usages
    double cost;
    3 usages
    String recipe;
    4 usages
    public pastry(String direction){ //reads file and gives values to variables
```

- Methods

Methods are used in my code in all of the classes to separate each part of the database into its purpose as a form of organization. If it weren't for methods, the

code would be hard to read which would make programming or debugging difficult. It is also used for repetition and helps with abstraction as it removes the need to rewrite code for each time a method is called.

```
6 usages
public class returnMenu {
    4 usages
    public returnMenu(JFrame frame, String text){
        JFrame leave = new JFrame( title: "Return");
        JPanel viewer = new JPanel();
        App exit = new App();
        viewer.setLayout(new BoxLayout(viewer,BoxLayout.Y_AXIS));
        JLabel title = new JLabel( text: "Which menu would you like to go to?");
        title.setSize( width: 1400, height: 80);
        viewer.add(title);
        JButton gui = new JButton( text: "Main Menu");
        gui.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                frame.dispose();
                leave.dispose();
                exit.GUI();
            }
        });
    }
};
```

This method is called 4 times and is around 50 lines of code. By having it in its own class and split into a method, it allows the abstraction of code to be left to one class and be called all over the program. This shows ingenuity because it reduces the amount of code needed.

- Objects

Objects are used to make taking specific parts of a pastry easier.

```

package com.codebind;

import java.io.File;
import java.io.FileReader;
import java.util.Scanner;

13 usages
public class pastry {
    // initializes object
    6 usages
    String Pname;
    3 usages
    String information;
    4 usages
    String ingredients;
    5 usages
    double cost;
    3 usages
    String recipe;
    4 usages
    public pastry(String direction){ //reads file and gives values to variables

```

My entire program uses these objects, as it is referenced in display, sorting/searching, and editing. This shows ingenuity because it reduces the amount of code needed each time a pastry is called.

- Polymorphism

Polymorphism is used in my code for the return menu class. The class will output a different amount of buttons based on the way the methods are called

```

public class returnMenu {
    4 usages
    public returnMenu(JFrame frame, String category){ //will run when user has picked specific category
        //initializes interface components
        JFrame returnFrame = new JFrame( title: "Return");
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel,BoxLayout.Y_AXIS));
        JLabel leaveLabel = new JLabel( text: "Which menu would you like to go to?");
        leaveLabel.setSize( width: 1400, height: 80);
        panel.add(leaveLabel);
        JButton gui = new JButton( text: "Main Menu");
        gui.addActionListener(e -> { //when user wants to go to main menu
            frame.dispose();
            returnFrame.dispose();
            App.GUI();
        });
        gui.setSize( width: 1400, height: 80);
        panel.add(gui);
        JButton categories = new JButton( text: "Categories of pastries");
        categories.addActionListener(e -> { //when user wants to go to the 5 categories
            frame.dispose();
            returnFrame.dispose();
            App.pastryList();
        });
        panel.add(categories);
    }
}

```

```

        JButton past = new JButton( text: "List of Pastries");
        past.addActionListener(e -> { //when user wants to go to specific list of pastries
            new buttons(category);
            frame.dispose();
            returnFrame.dispose();
        });
        past.setSize( width: 1400, height: 80);
        past.setSize( width: 1400, height: 80);
        panel.add(past);
        JButton cancel = new JButton( text: "Cancel");
        cancel.addActionListener(e -> returnFrame.dispose()); //cancels and the user can resume their last frame
        categories.setSize( width: 1400, height: 80);
        panel.add(categories);
        panel.add(cancel);
        returnFrame.add(panel);
        returnFrame.pack();
        returnFrame.setVisible(true);
    }
}

1 usage
public returnMenu(JFrame frame){ //used when specific pastry category isn't necessary
    //initializes components
    JFrame returnFrame = new JFrame( title: "Return");
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel,BoxLayout.Y_AXIS));
    JLabel leaveLabel = new JLabel( text: "Which menu would you like to go to?");
    leaveLabel.setSize( width: 1400, height: 80);
    panel.add(leaveLabel);
    JButton displayButton = new JButton( text: "Main Menu");
    displayButton.addActionListener(e -> { //returns to main menu

```

Based on the context that the return menu is called in, with or without a category, it will display different buttons. Polymorphism is used here to achieve the same end goal, a menu that allows the client to switch between windows, while keeping it simple and not confusing.

- Recursion

Recursion is used in my add class and allows the user to repeatedly add pastries to the same category.

```

public class add {
    public add(String category){
        //initializes interface

```

```

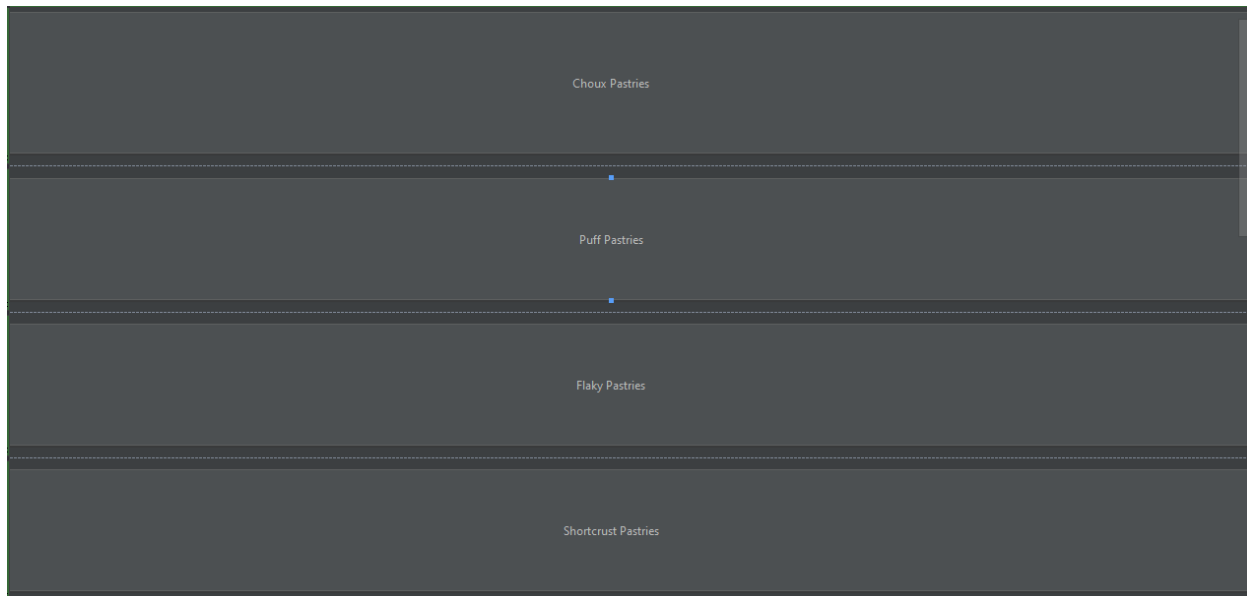
JButton exit = new JButton( text: "No");
stay.addActionListener(e1a -> new add(category)); //if user wants to add another file (recursion)
exit.addActionListener(e1b -> { //when user is done adding files
    repeat.dispose();
    App.GUI();
});

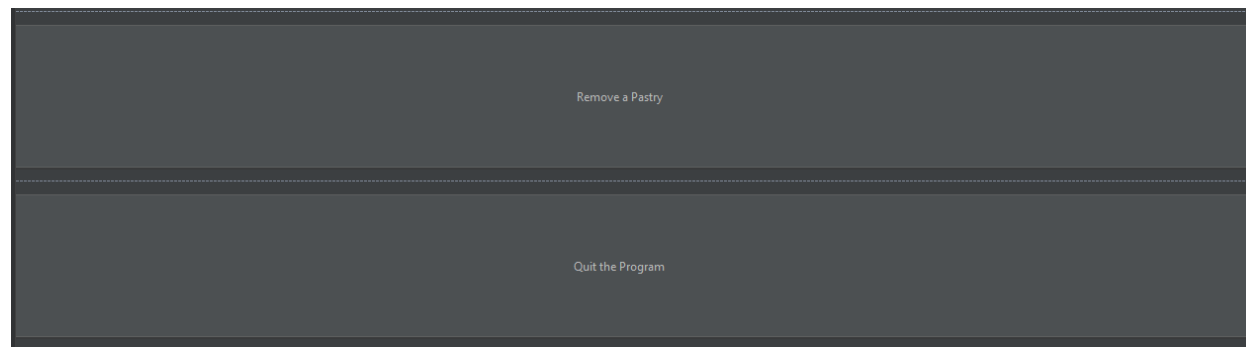
```

Without recursion, the code would have to call back to another class and start the process over, making it less user friendly and a longer process. Using recursion cuts down on this process by repeating the entire process but quicker and in less lines of code. This shows ingenuity because it reduces multiple steps by repeating itself.

- GUI

One of my defined success criteria was to make a button-based user interface to reduce complexity and make it easier to navigate.





The GUI is meant to be user-oriented and achieves my success criteria of providing an easy access and easy to read interface. This shows complexity by introducing a new module to work with that causes the program to run differently yet creates a more efficient outcome for the client.

- Files

Files are the center point of my database, as it is used to meet every success criteria. Files store the data and all of my coding techniques are based on the files, along with the user interface. Without the usage of files, my program wouldn't be considered a database, only a user interface that doesn't achieve the success criteria.

```
String name = "C:\\Users\\691030\\IdeaProjects\\Swing\\text\\" + category + "\\ " + fName + ".txt";
File newF = new File(name);
try {
    if (newF.createNewFile()) {
        FileWriter edited = new FileWriter(name);
        String body = "Name:" + "\n" + fName +
            "\nInformation:" + "\n" + info +
            "\nIngredients:" + "\n" + ingr +
            "\nRecipe:" + "\n" + recipe;
        edited.write(body);
        edited.close();
    }
}
```

With files, I can accomplish most of my testing plan by testing if files accurately save with the correct data and name and if they delete the file. This shows ingenuity because it efficiently stores data in a specified location and lets the code save in real time and add to my database. This also is complex because my code implements three different imports based around files, File, FileReader, FileWriter, along with multiple methods that stem from these classes.

- Conditionals

Conditionals are a huge centerpiece of a specific part of my code. The display to view the list of pastries would not be as simple if it weren't for conditional statements. Conditional statements were used to decide if the user wanted to view, edit, or delete a pastry.

```

public void pastries(){
    getListFrame().dispose();
    try{
        if ("add".equals(condition)) {
            new add(category);
        }else {
            new buttons(category);
        }
    }
}

```

In this case, there are only two things the code can do, go to the add class or the buttons class. Primarily four methods call this class and all except for one of them will go to the buttons class, which is why the if and else statements are important. They are used to differentiate between what method will move onto the add class and which will move onto the buttons class. My conditional statements are used to prevent the repetition of code based on what the user wants. This shows ingenuity because, like the method, the code can input data and accounts for variables like the lengths of a list and runs a loop for each element in a specific list.

- Case Switch

In cases where there are multiple possible ways the call will go to. I resorted to using case switch statements to reduce code and confusion.

```

switch (App.condition) { //based on conditions from App class
    case "display" -> displayed(buttonList, category, mainFrame);
    case "delete" -> deleted(buttonList, mainFrame);
    case "edit" -> edited(buttonList, mainFrame);
}

```

In the example, based on what the user previously picked, the code will follow one of three paths. This shows ingenuity because it reduces the amount of code needed, because if conditionals were used, the code would be nearly double the amount of lines of code, and therefore creating less confusion. This also shows

complexity because it is a higher level of conditional-based programming and strays away from the curriculum.

- Repetition - **DB1a**

Repetition is the way how my buttons are created and without using a loop, it would cause my program to not be as simple.

```
public static void displayed(ArrayList<String> buttonList, String category, JFrame frame){ //if client chose to display pastries
    int size = buttonList.size();
    JButton[] buttons = new JButton[size];
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.PAGE_AXIS));
    for (int i = 0; i < buttonList.size(); i++) {
        buttons[i] = new JButton(buttonList.get(i));
        buttons[i].setActionCommand(buttonList.get(i));
        buttons[i].addActionListener(e -> {
            String choice = e.getActionCommand();
            display show = new display();
            show.displayInfo(category, choice);
            frame.dispose();});
        panel.add(buttons[i]);
    }
    JButton random = new JButton( text: "Would you like to find a random pastry from this category?");
    random.addActionListener(e -> { // random button
        Random rand = new Random();
        int index = rand.nextInt(buttonList.size());
        String choice = buttonList.get(index);
        display show = new display();
        show.displayInfo(category, choice);
        frame.dispose();});
    panel.add(random);
    buttonAdd(frame, panel, buttonList);
}
```

In this, the program takes the values of an arraylist and turns each element into a button. The use of repetition allows for any amount of elements to all have the same properties and allow my program to flow.

- Try Catch Exceptions

A major component of my code, the editing and add feature, requires the try and catch to run an imported method, in this case filewriter.

```

if (!(pastryName.getText().equals(pastry))){ //if file is given a new name, deletes the old file
    newName += category + "\\ " + pastry + ".txt";
    File deleteFile = new File(newName);
    deleteFile.delete();}
try { //part that edits the file
    FileWriter editWriter = new FileWriter(newFile);
    String body = "Name:"+"\n"+pastryName.getText()+
        "\nInformation:"+"\n"+addInfo.getText()+
        "\nIngredients:"+"\n"+ingredients.getText()+
        "\nCost per serving ($):"+"n"+cost.getText()+
        "\nRecipe:"+"\n"+recipe.getText();
    editWriter.write(body);
    editWriter.close();
} catch (IOException ex) { //if an error catches through
    new error();}

```

In this case, I imported a new method called FileWriter and if I didn't use a try catch exception statement, the FileWriter would be invalid the method throws an exception. As detailed in my testing plan, this is used for correct user input as if the user does not input the correct data, an error occurs.

- Algorithms:
- Sorting/searching - CS2

This accomplishes part of my testing plan as I can use it to search for ingredients and make sure that my program is properly searching for the correct ingredient and displaying all of the recipes that have the ingredient.

```

public void name(String input, ArrayList<String> pastryList, String category){
    ArrayList<String> newPastryList = new ArrayList<>();
    List<String> result = pastryList.stream() //converts all parts of code to lowercase
        .map(String::toLowerCase).toList();
    input = input.toLowerCase();
    for (String pastry:result) { //compares each pastry to see if user input matches the pastry name
        Pattern pattern = Pattern.compile(input);
        Matcher match = pattern.matcher(pastry);
        if (match.find()) {newPastryList.add(pastry);}}
    callback(newPastryList,category);}

```

```

public void ingredient(String input, ArrayList<String> list,String category){
    ArrayList<String> newPastryList = new ArrayList<>();
    Pattern pattern = Pattern.compile(input); // takes input to compile
    for(String pastry:list){ // compares each pastry's ingredients to the input
        pastry specificPastry = new pastry( direction: "text\\" + category + "\\" +pastry + ".txt");
        Matcher match = pattern.matcher(specificPastry.ingredients); //checks if the input is in the list of ingredients
        if(match.find()){newPastryList.add(pastry);}
    }
    callback(newPastryList,category);}

```

```

public void callback(ArrayList<String> newPastryList, String category){
    ArrayList<pastry> coll = new ArrayList<>();
    if(!nameSort){ //sorts by cost
        for(String pastry:newPastryList){
            pastry pastry1 =new pastry( direction: "text\\" + category + "\\" +pastry + ".txt");
            coll.add(pastry1);
        }
        coll.sort(new pastryCompare()); // sorts by cost
    }else{ //sorts by name
        Collections.sort(newPastryList);
        for(String pastry:newPastryList){
            pastry pastry1 =new pastry( direction: "text\\" + category + "\\" +pastry + ".txt");
            coll.add(pastry1);
        }
    }
    newPastryList.clear(); //clears new list to use later
    if(!lowest) { //reverse sort, can't use collections comparator for costs so resort to this
        for(int i = coll.size()-1;i>=0;i--){
            newPastryList.add((coll.get(i)).Pname);}
        }else{for(pastry pastry:coll){newPastryList.add(pastry.Pname);}}
    search.dispose();
    mainFrame.dispose();
    JFrame pastryFrame = new JFrame( title: "Filtered list of " + category + " pastries");
    switch (App.condition) { //calls button display class to display new list of pastries
        case "display" -> buttons.displayed(newPastryList, category, pastryFrame);
        case "delete" -> buttons.deleted(newPastryList, pastryFrame);
        case "edit" -> buttons.edited(newPastryList, pastryFrame);
    }
}

```

```

public static class pastryCompare implements Comparator<pastry> {
    @Override
    public int compare(pastry p1, pastry p2) {
        int costCompare = Double.compare(p1.cost, p2.cost);
        if (costCompare != 0) {
            return costCompare;
        }
        return p1.Pname.compareTo(p1.Pname);
    }
}

```

- Adding and Deleting - **DB2**

Accomplishes part of testing plan and success criteria by making sure that the program properly adds and deletes files.

```

try {
    if (newF.createNewFile()) { //if the file is made
        FileWriter edited = new FileWriter( fileName: "text\\" + App.category + "\\" + nameField.getText() + ".txt");
        String body = "Name:" + "\n" + nameField.getText() +
            "\nInformation:" + "\n" + addInfo.getText() +
            "\nIngredients:" + "\n" + ingredients.getText() +
            "\nCost per serving ($):" + "\n" + cost.getText() +
            "\nRecipe:" + "\n" + recipe.getText();
        edited.write(body);
        edited.close();
        repeat.setVisible(true); //only sets visible if code works, or else will call on error class
    } else { //if the file isn't made, error pops up
        confirm.dispose();
        new error();
    }
} catch (IOException ex) { //if try doesn't work, error class is called
    confirm.dispose();
    new error();
}
confirm.dispose();
show.dispose();
});
confirmCancel.addActionListener(e2 -> confirm.dispose()); //if user doesn't want to confirm their pastry

```

```

public class delete {
    1 usage
    public delete(String category, String name, JFrame frame){
        File file = new File( pathname: "text\\" + category + "\\" +name + ".txt"); //calls file
        file.delete(); //deletes
        frame.dispose(); //disposes of last frame
        //start of initializing interface
        JFrame repeat = new JFrame( title: "Return");
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel,BoxLayout.Y_AXIS));
        JLabel leave = new JLabel( text: "Would you like to exit?");
        JButton confirm = new JButton( text: "Yes");
        JButton deny = new JButton( text: "No");
        confirm.addActionListener(e -> new returnMenu(repeat, category));
        deny.addActionListener(e -> {
            App main = new App();
            main.pastries();
        });
        panel.add(leave);
        panel.add(confirm);
        panel.add(deny);
        repeat.add(panel);
        repeat.pack();
        repeat.setVisible(true);
    }
}

```

- Editing - DB3

It accomplishes the editing part of the testing plan, making sure that all changes made, including the new name, are properly changed in the source files.

```

confirm.addActionListener(e1 -> { //if user chooses to confirm
    confirmWindow.dispose();
    String name = "text\\";
    String newName = name;
    name += category + "\\" + pastryName.getText() + ".txt";
    File newFile = new File(name);
    if (!(pastryName.getText().equals(pastry))) { //if file is given a new name, deletes the old file
        newName += category + "\\" + pastry + ".txt";
        File deleteFile = new File(newName);
        deleteFile.delete();
    }
    try { //part that edits the file
        FileWriter editWriter = new FileWriter(newFile);
        String body = "Name:"+"\n"+pastryName.getText()+
            "\nInformation:"+"\n"+addInfo.getText()+
            "\nIngredients:"+"\n"+ingredients.getText()+
            "\nCost per serving ($):"+"n"+cost.getText()+
            "\nRecipe:"+"\n"+recipe.getText();
        editWriter.write(body);
        editWriter.close();
    } catch (IOException ex) { //if an error catches through
        new error();
    }
}

```

Word Count: 1105